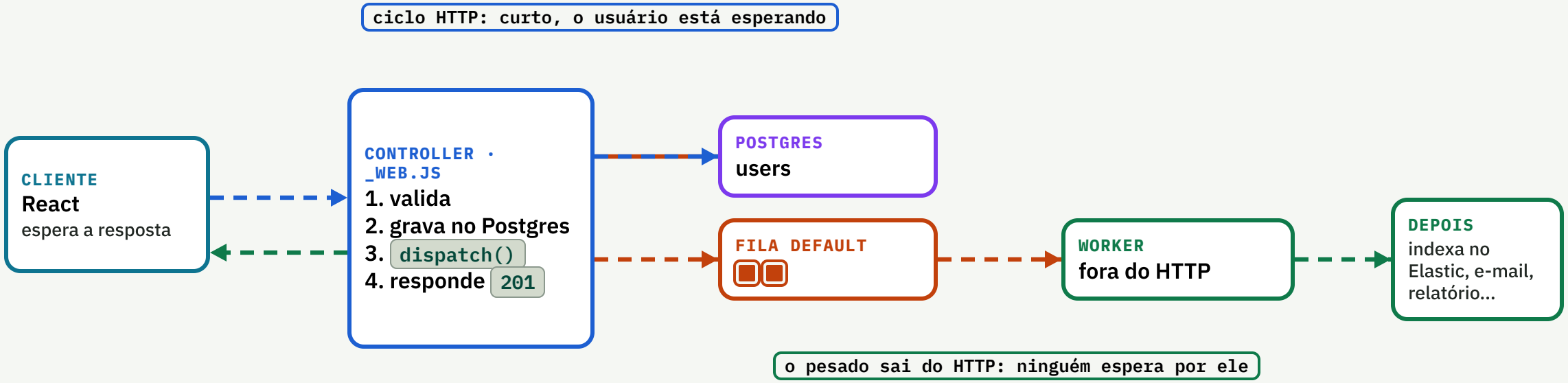
UNIFAAT 2026.2

FRONTEND

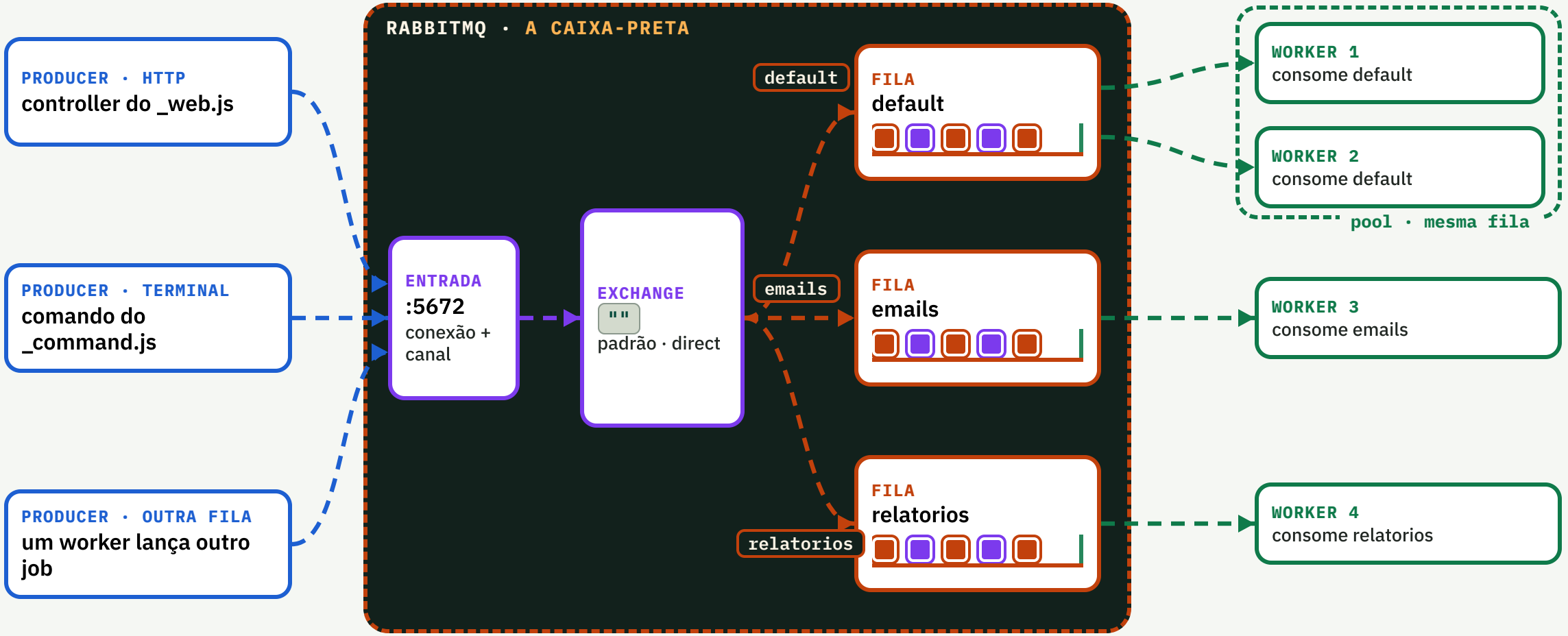
10 - Mensageria e Websockets

Prof. Luan Lourenço

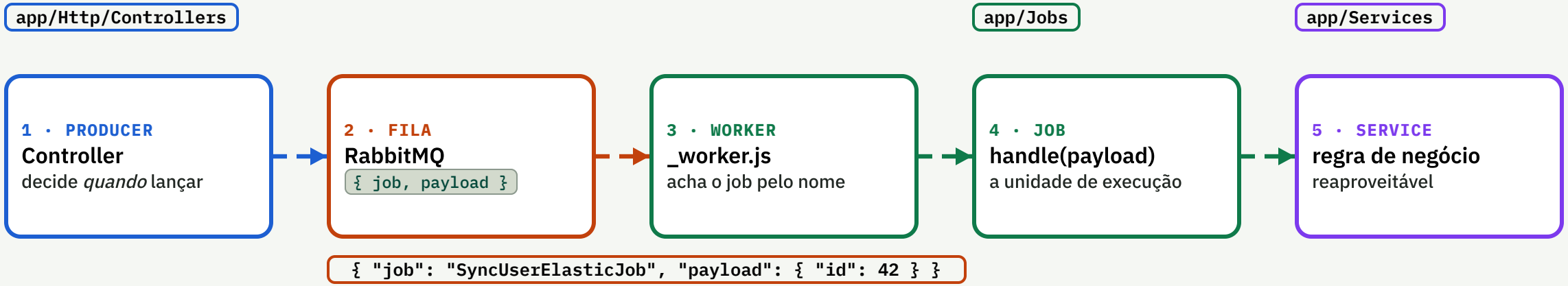
Ciclo HTTP leve, trabalho pesado no worker



Por dentro do RabbitMQ: a caixa-preta

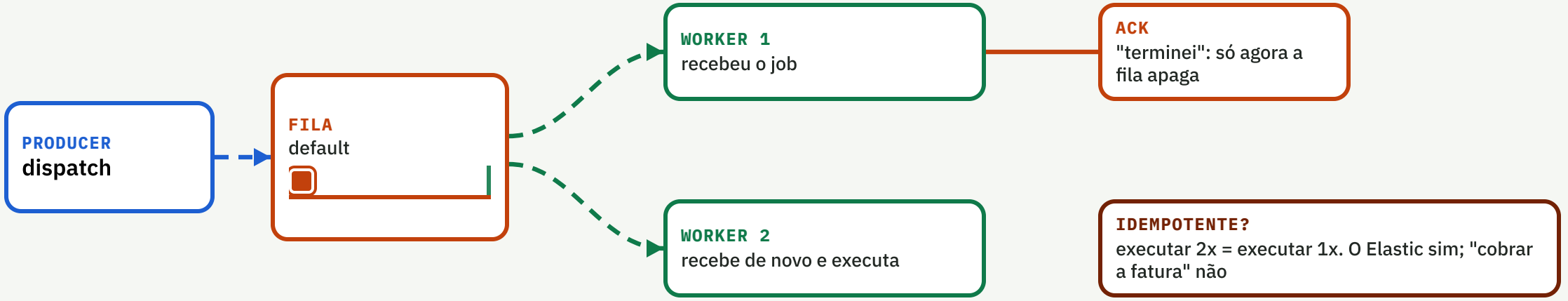


O `job` é a unidade de execução



Por isso o payload leva só o `id`: o service lê o **estado atual** no Postgres na hora de executar.

Um job, um worker: ack e idempotência



Entrega a um só é propriedade da **fila**. Idempotência é propriedade do **job**.

Escalar jobs: mais workers na mesma fila

PRODUCER · LANÇA 2 JOBS POR SEGUNDO



2 jobs esperando na fila

cada worker consome ~0,7 job por segundo

WORKERS PENDURADOS NA MESMA FILA

worker 1 consome | worker 2 consome | worker 3 consome | worker 4

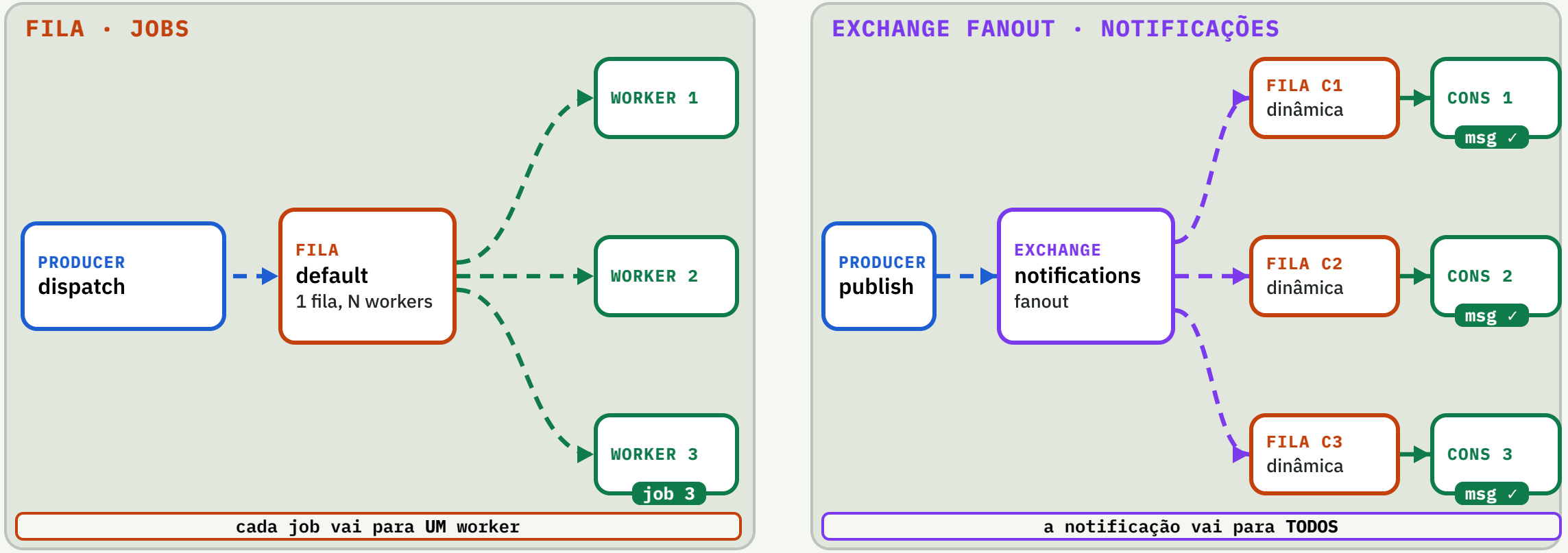
– worker | + worker

3 workers fazem 2,1/s: a fila ESVAZIA

```
docker compose up -d --scale nodeworker-container=3
```

Os workers **competem** pelas mensagens (*competing consumers*). O RabbitMQ **empurra** o job: não existe *polling*.

Fila divide, exchange divulga



Job = **ordem** ("faça isto"), um executor. Notificação = **aviso** ("isto aconteceu"), vários interessados.

O exchange roteia pelos bindings

direct chave igual

`rk = emails`

fila a	**bind: emails**
fila b	bind: relatorios

Entrega às filas cujo **binding** é **igual** à routing key.

fanout broadcast

`rk ignorada`

fila a	**bind: –**
fila b	**bind: –**
fila c	**bind: –**

Copia para **todas** as filas ligadas. É o que usamos nas notificações.

topic padrão

`rk = chat.sala1`

fila a	**bind: chat.***
fila b	**bind: chat.#**
fila c	bind: pedido.*

A routing key casa com um **padrão** (`*` uma palavra, `#` várias).

default ("") direct, implícito

`rk = default`

fila default	**bind: default**
fila emails	bind: emails

Já existe. Toda fila é ligada a ele pelo **nome dela**: por isso o `sendToQueue` funciona.

`sendToQueue("default", msg)` é só um atalho para `publish("", "default", msg)`: **toda** mensagem passa por um exchange.

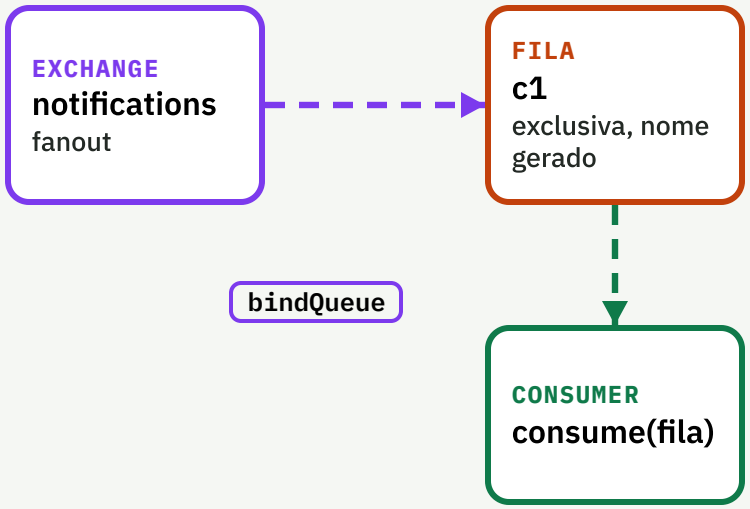
Configurando o exchange na conexão

```
await ch.assertExchange("notifications",
"fanout")
// cria o exchange (se já existe, tudo bem)

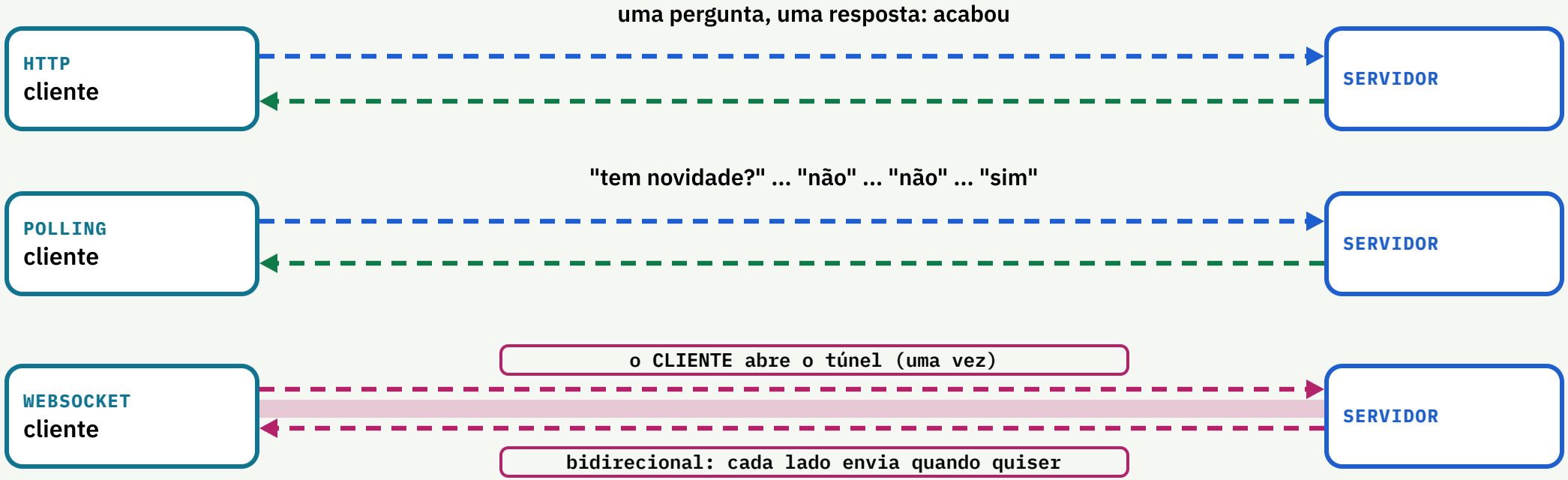
const { queue } = await ch.assertQueue("", {
exclusive: true })
// fila SÓ deste processo; some ao desconectar

await ch.bindQueue(queue, "notifications", "")
// binding: o fanout ignora a routing key

await ch.consume(queue, onMessage)
// producer: ch.publish("notifications", "",
msg)
```



O cliente sempre vai atrás do servidor



HTTP vira WebSocket: o handshake e o Network

CLIENTE → SERVIDOR · HTTP COMUM

```
GET /socket HTTP/1.1
Host: localhost:8080
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhl...ZQ==
Sec-WebSocket-Version: 13
```

SERVIDOR → CLIENTE

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pP...xOo=
```

a MESMA conexão TCP agora troca frames

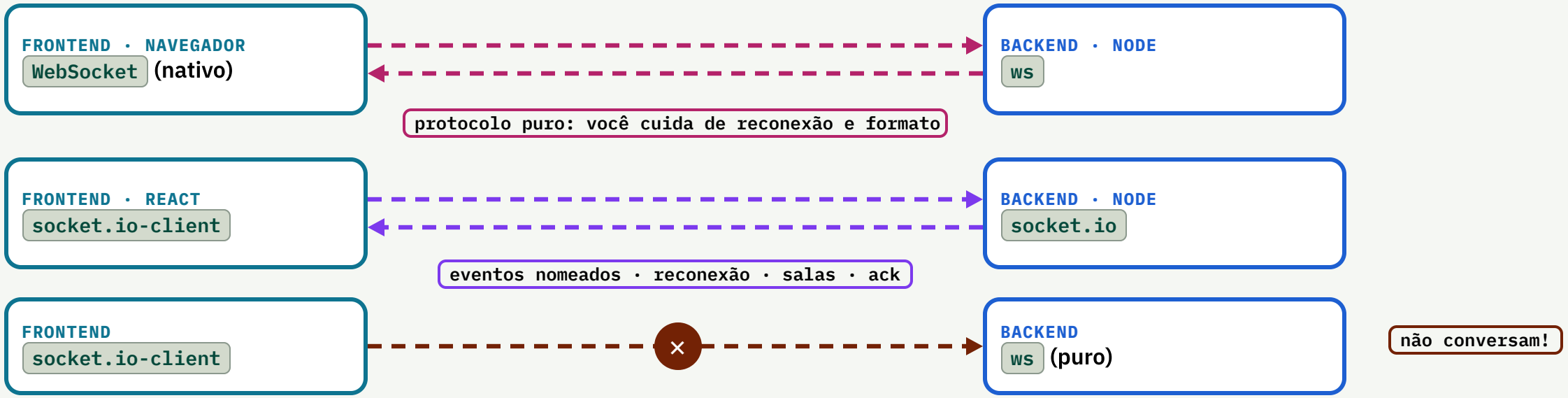
Elements | Console | **Network**

Fetch/XHR · JS · **WS** socket · **101**

Headers **Messages**

↑	chat:message {"text":"oi"}
↓	chat:message {"name":"Ana"}
↓	chat:message {"name":"Beto"}
↑	chat:message {"text":"bora"}
↓	chat:message {"name":"Caio"}

Uma lib em cada ponta, do mesmo sabor



O Socket.IO começa por *long polling* e **sobe** para WebSocket: no Network você vê o `polling` e depois a linha `101`.

Um novo entrypoint: o Event é o job do websocket

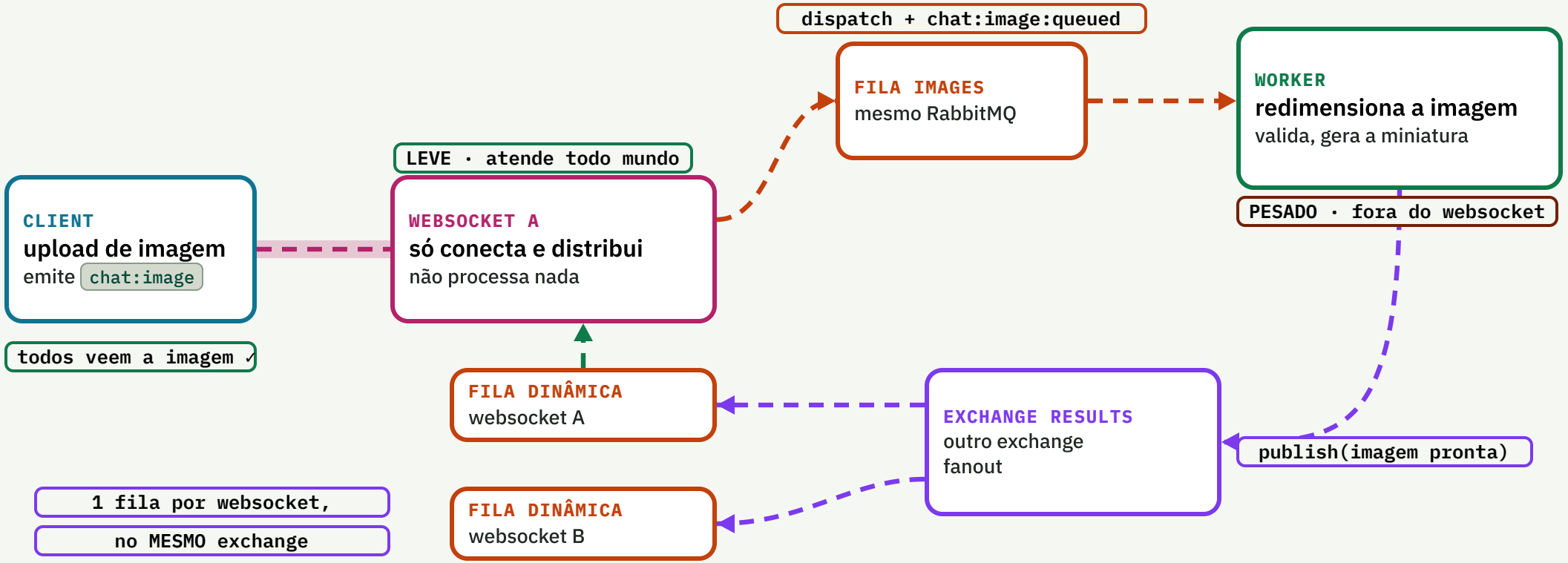
Entrypoint	O que sobe	Unidade de execução	Quem produz	Parâmetro
`_web.js`	HTTP (Express)	Controller	o navegador (request)	`PORT`
`_command.js`	comando do terminal	Command	você, no terminal	nome do comando
`_worker.js`	consumer de **fila**	**Job** · `app/Jobs`	controller, comando, worker	`--queue`
`_websocket.js`	consumer de **notificações** + túneis	**Event** · `app/Events`	o **client**, que abre a conexão	`--exchange`

Job **producer → fila → worker**
uma **ordem**, um executor

Event **client → túnel → websocket → exchange → todos**
um **aviso** nomeado, `chat:message`

Outro **container** (`nodewebsocket-container`): o websocket mantém centenas de conexões abertas e **só conecta e distribui eventos**. O pesado vai para um **worker**.

O websocket só distribui: o pesado vai para o worker



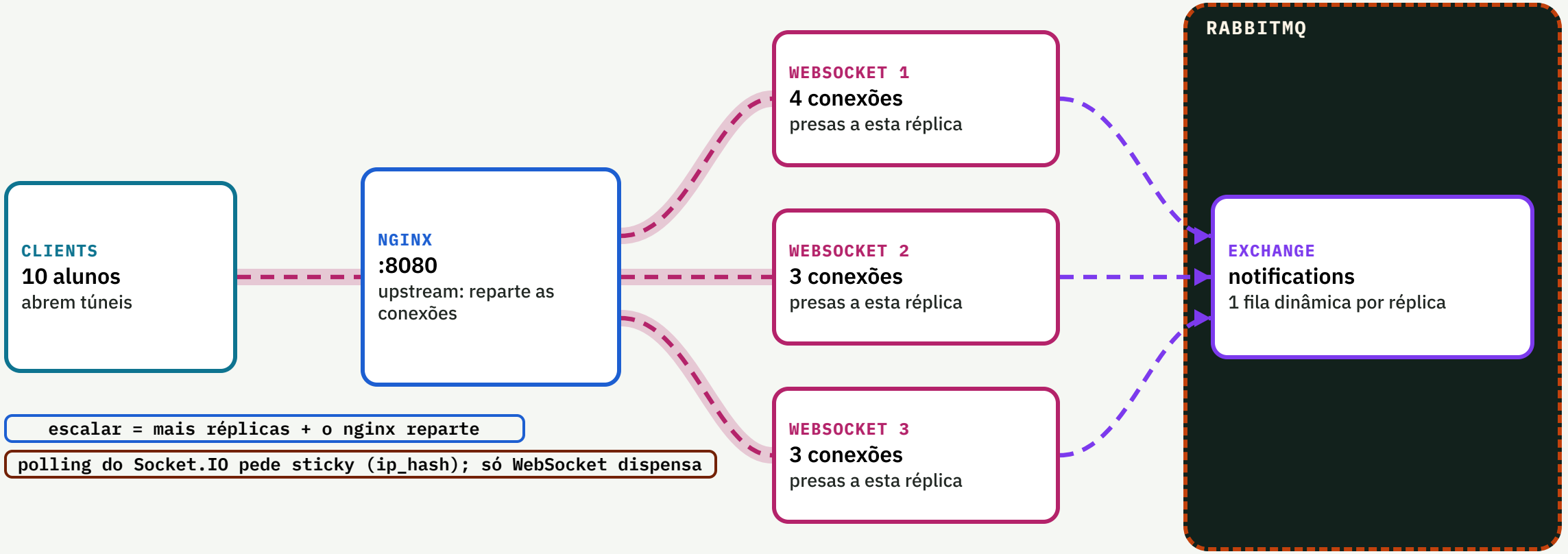
Websocket e worker são containers diferentes no **mesmo RabbitMQ**: o websocket cuida das conexões, o worker cuida da conta.

Worker × websocket: um fluxo, dois papéis



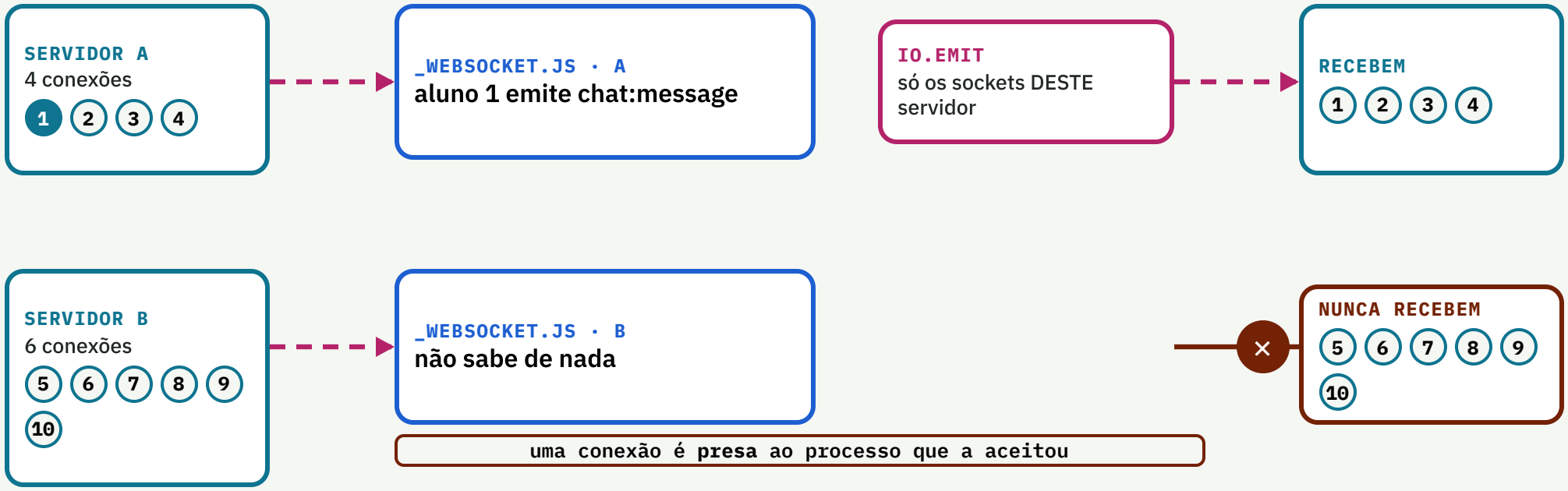
O worker faz o pesado; o websocket segura as conexões. Os dois compartilham um fluxo só, nos dois sentidos.

O nginx também escala as conexões



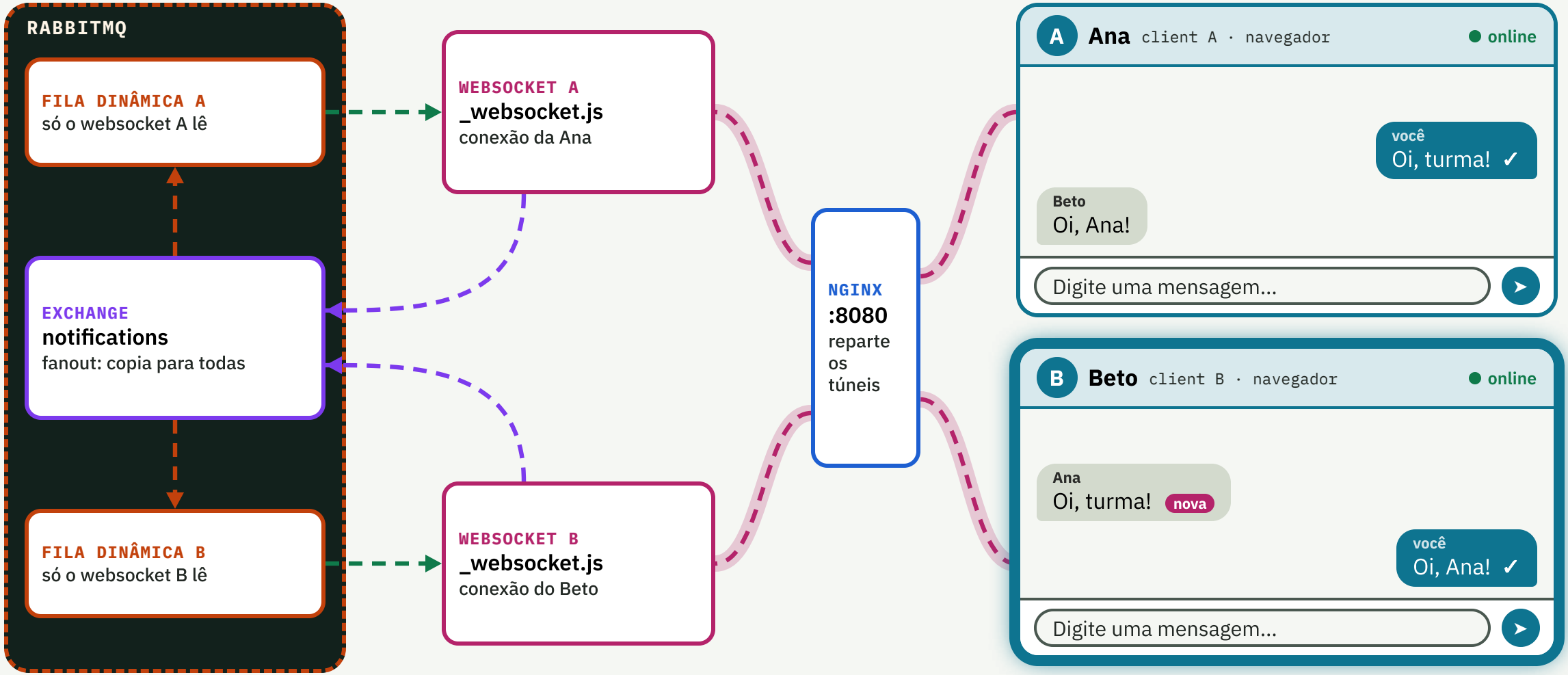
Mais websockets só ajudam se o **nginx** reparte os túneis entre eles; o **exchange** garante que a mensagem chegue a todos.

10 alunos, 2 servidores: quem recebe?



Falta um jeito de a mensagem de **um** servidor chegar a **todos** os outros: **notificação**, não job.

O chat em ação: do clique da Ana à tela do Beto



TF da Aula 10: a definir

- **Enunciado:** a definir
- **Entrega:** como nas aulas anteriores, por **Pull Request** (branch `RA-tf-10`); o link do TF será o diretório `aulas/10` no GitHub
- **Prazo:** a definir